РАЗРАБОТКА ДИЗАЙНА ВЕБ-ПРИЛОЖЕНИЙ

DesignFolio

Платформа для публикации и просмотра дизайнерских работ

ФИО: Пятков Андрей Владимирович

Группа: ИС 3-01

Проблема и аудитория

Платформа соединяет авторов работ и тех, кто ищет идеи или специалистов.

ПРОБЛЕМА

Работы разбросаны по разным площадкам

Сложно собрать проекты в единую витрину и удобно находить авторов по направлению.

единый каталог портфолио

ЦЕЛЕВАЯ АУДИТОРИЯ

01 Автор работ

Студент или начинающий дизайнер хочет публиковать проекты, показывать навыки и получать обратную связь.

02 Посетитель / работодатель

Ищет работы по категориям, изучает страницу проекта и профиль автора.

Ключевые функции

Функции сгруппированы вокруг просмотра, публикации и управления контентом.

01

Находить

Каталог работ, поиск, категории и сортировка.

02

Изучать

Страница проекта, изображения и профиль автора.

03

Публиковать

Создание и редактирование работ, загрузка изображений.

04

Общаться

Регистрация и вход, лайки и комментарии.

05

Управлять

Профиль, личный кабинет и административная модерация.

Структура сайта

Основной путь: каталог - отдельная работа - профиль автора.



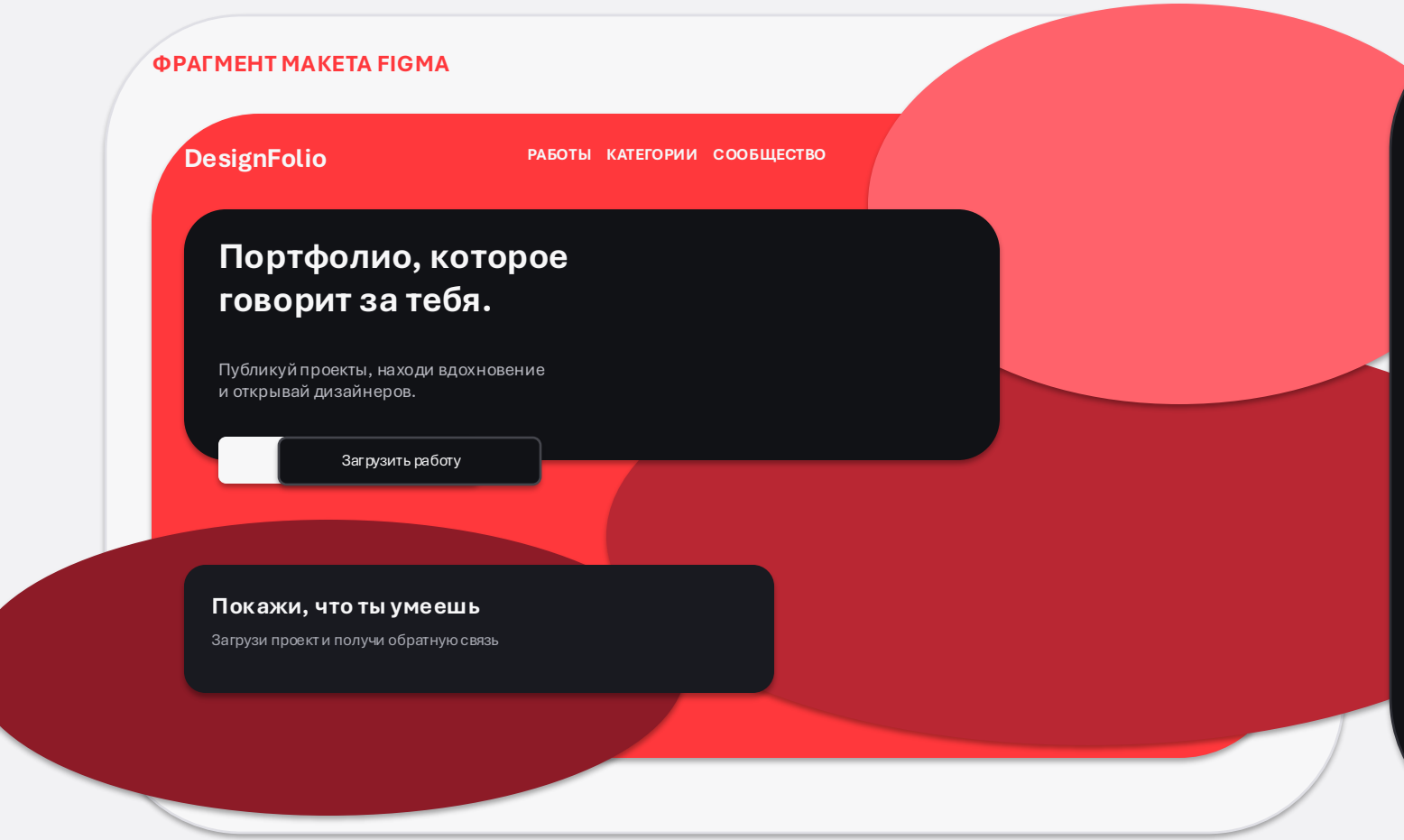
Слои проекта

Frontend	React + Vite
Backend	FastAPI
Данные	PostgreSQL + Storage

Визуальный язык проекта

Контраст помогает отделить контент от основных действий.

ФРАГМЕНТ МАКЕТА FIGMA



ПАЛИТРА

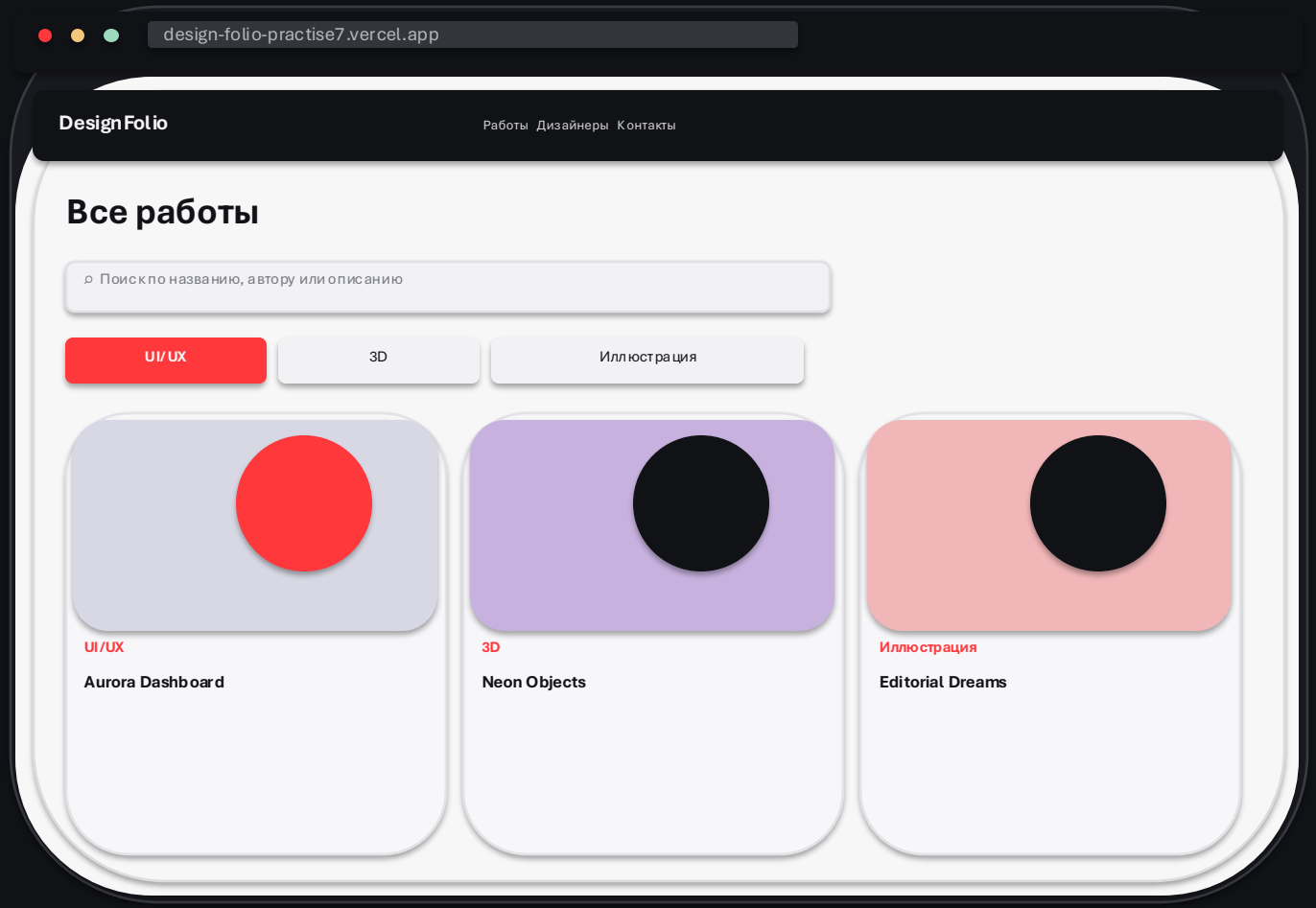
	Акцент	#FF383C
	Глубокий красный	#B92733
	Фон / панели	#111216
	Светлый текст	#F7F7F8

Принципы

Крупные заголовки • высокий контраст • акцентный цвет на CTA

Основной сценарий пользователя

Вживую пройти один короткий путь от поиска до страницы проекта.



СЦЕНАРИЙ ДЕМОНСТРАЦИИ

01 **Открыть каталог**
/works

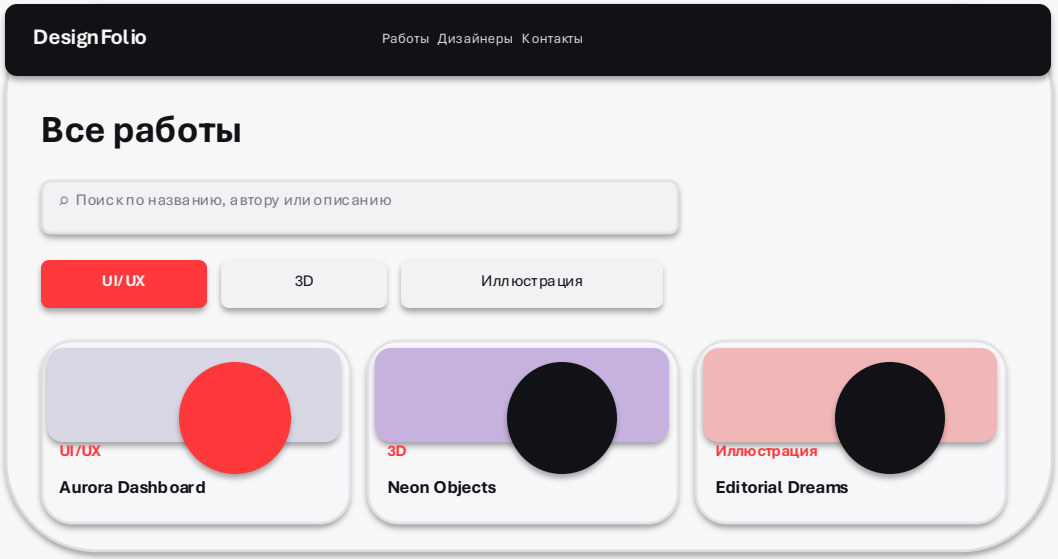
02 **Найти категорию или ввести запрос**
поиск / фильтр

03 **Открыть работу и профиль автора**
работа → дизайнер

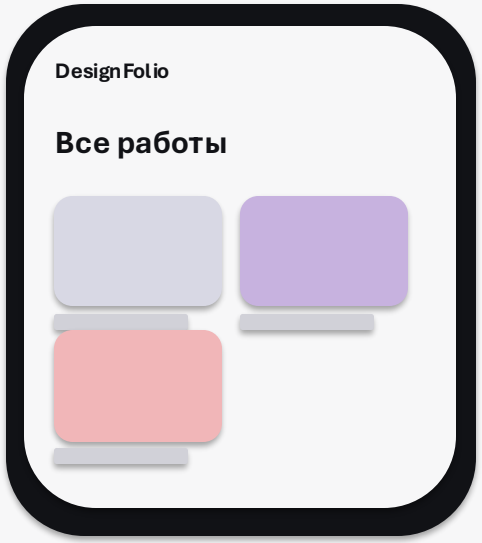
Ссылка для показа: design-folio-practise7.vercel.app

Проверка адаптивности

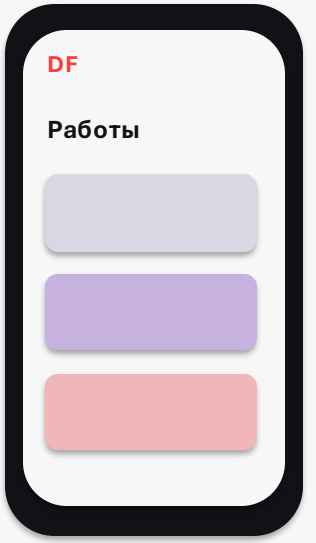
DESKTOP



TABLET



PHONE



Проверка в Edge: F12 → эмуляция устройства

Сравнить сетку, меню, кнопки и отсутствие горизонтальной прокрутки.

Frontend: единый клиент API

Файл: designfolio/frontend/src/api/client.js

PY

```
if (token) headers.Authorization =
  `Bearer ${token}`;
const response = await fetch(API + path, {
  ...options, headers,
});
const body = response.status === 204
  ? null
  : await response.json().catch(() => ({}));
if (!response.ok) {
  throw new Error(body?.detail || "Ошибка");
}
return body;
```

Зачем нужен helper?

- Общий fetch для всех страниц.
- Передаёт Bearer-токен после входа.
- Разбирает JSON и HTTP-ошибки.

Backend: пароль и JWT

Файл: designfolio/backend/app/routers/auth.py

PY

```python
pwd_context = CryptContext(
  schemes=["bcrypt"],
  deprecated="auto",
)

def hash_password(password: str) -> str:
  return pwd_context.hash(password)

def create_access_token(user_id: int) -> str:
  expire = datetime.now(timezone.utc)
  expire += timedelta(hours=24)
  return jwt.encode(
    {"sub": str(user_id), "exp": expire},
    settings.secret_key, algorithm="HS256"
  )
```

Что здесь важно?

- В БД лежит bcrypt-хеш, не пароль.
- После входа создаётся подписанный JWT.
- В токене есть ID и срок: 24 часа.

Backend: безопасная загрузка изображений

Файл: designfolio/backend/app/routers/images.py

PY

```python
ALLOWED = {
  "image/jpeg": ".jpg",
  "image/png": ".png",
  "image/webp": ".webp",
}
MAX_SIZE = 5 * 1024 * 1024
if (
  work.author_id != current_user.id
  and current_user.role != "admin"
):
  raise HTTPException(status_code=403)
if file.content_type not in ALLOWED:
  raise HTTPException(status_code=400)
data = await file.read()
if len(data) > MAX_SIZE:
  raise HTTPException(status_code=413)
```

Проверки до сохранения

- Загрузить может автор или администратор.
- Разрешены JPEG, PNG и WebP.
- Лимит файла — 5 МБ; хранение в Storage.

Почему PostgreSQL + Supabase

Данные проекта связаны между собой, поэтому подходит реляционная модель.

ВЫБРАНО

PostgreSQL

- Внешние ключи и связи.
- Уникальность для логина, тегов и связок.
- Удобно работать через SQLAlchemy и Alembic.

АЛЬТЕРНАТИВА

SQLite

- Простой локальный старт.
- Файл базы проще переносить.
- Для облачного многопользовательского проекта менее удобно.

АЛЬТЕРНАТИВА

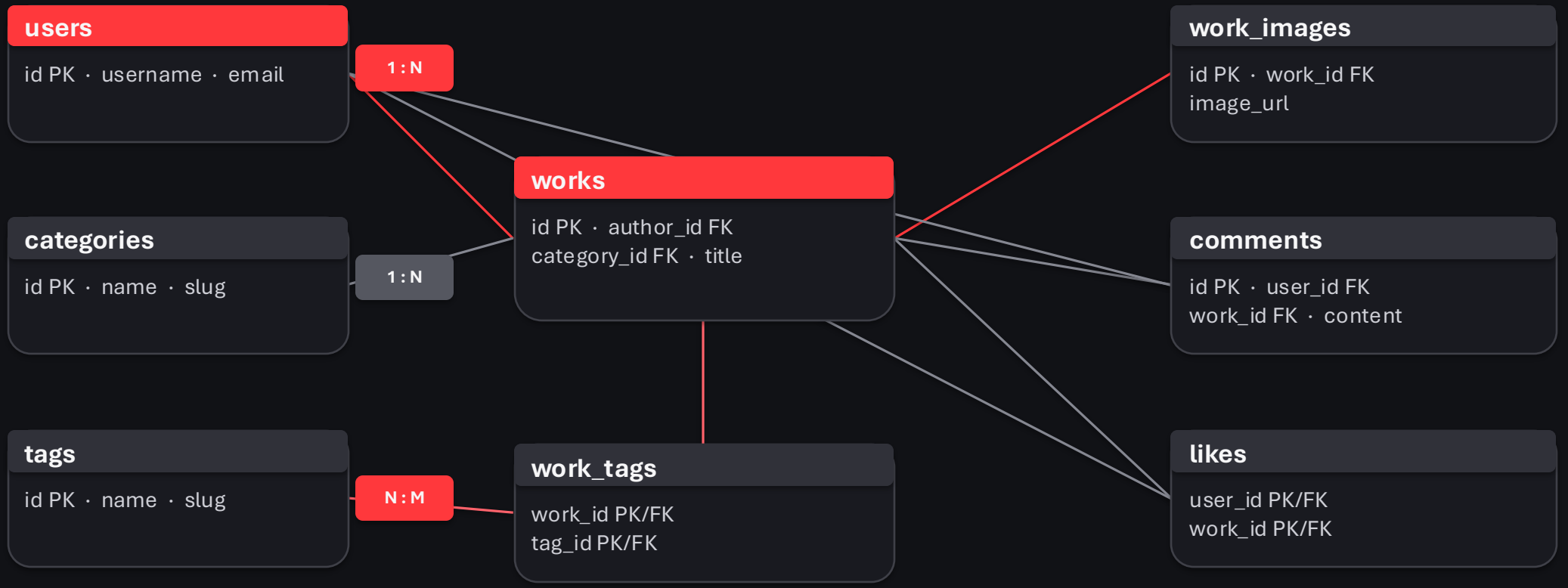
MongoDB

- Гибкая документная модель.
- Но связи автора, работ и реакций здесь явные.
- Реляционная схема естественнее для этих сущностей.

Supabase = управляемый PostgreSQL + Storage для файлов проекта

ER-диаграмма: основные сущности

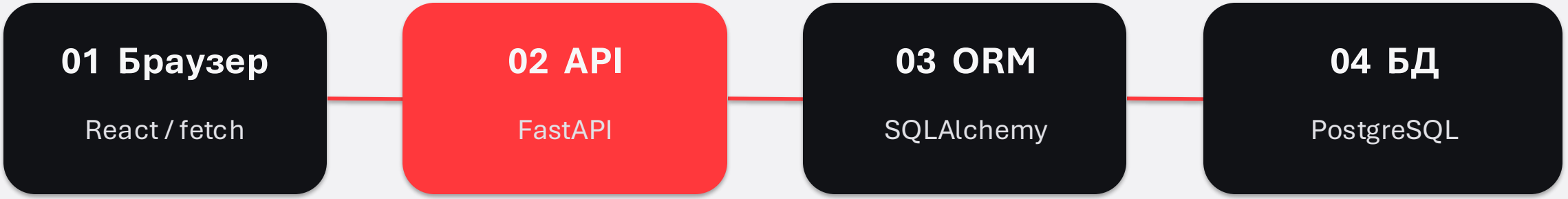
В схеме 8 основных таблиц; связи закреплены внешними ключами.



likes и work_tags используют составные ключи — повторная связь не дублируется.

Как данные доходят до страницы

Браузер не подключается к PostgreSQL напрямую — между ними работает API.



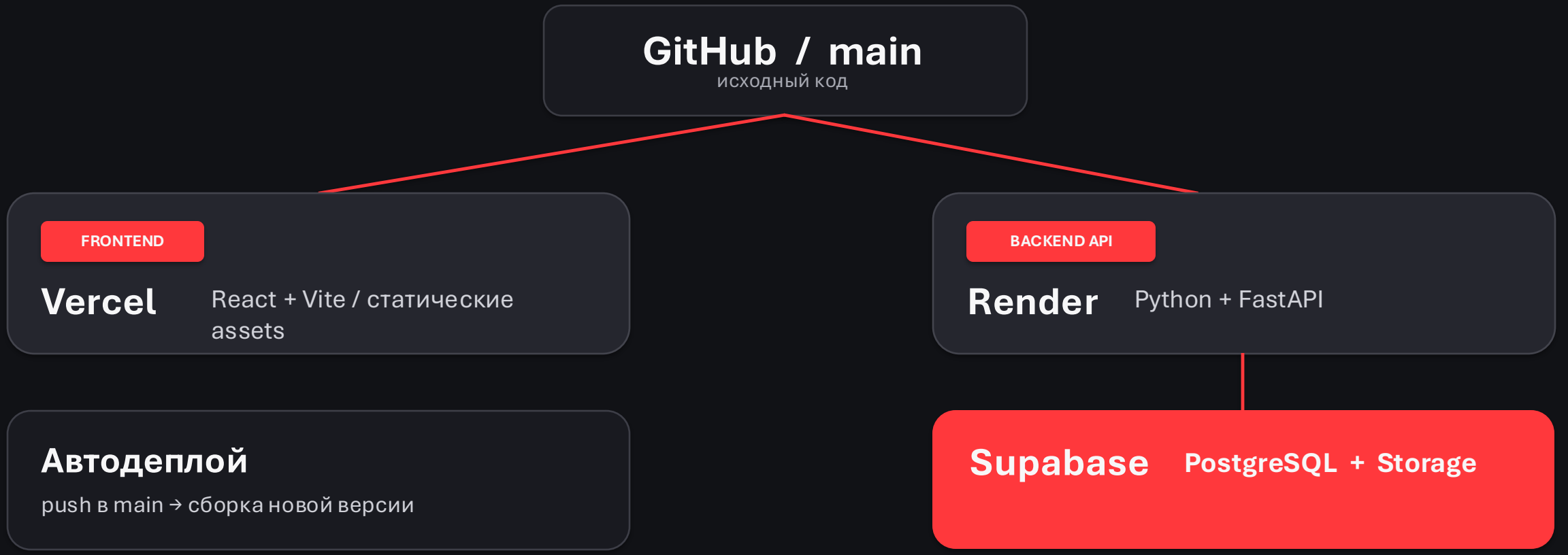
Запрос → проверка → запрос к данным → JSON-ответ

Загрузка изображения: FastAPI → Supabase Storage

В БД сохраняется URL изображения

Где размещены части приложения

Исходный код — GitHub; исполнение frontend, API и базы — на отдельных сервисах.



Важно: Render на бесплатном тарифе может «засыпать», поэтому первый запрос иногда дольше.

Что получилось и что улучшать дальше

РЕАЛИЗОВАНО

- Каталог, поиск, категории и сортировка
- Страницы работ и профили дизайнеров
- Регистрация, вход, JWT и управление профилем
- Публикации, изображения, лайки и комментарии
- Административные маршруты для модерации

ДАЛЬШЕ

Технические сложности

Настроить взаимодействие frontend и API, CORS, переменные окружения и удалённое хранение изображений.

Возможные улучшения

Автоматические проверки, тщательная проверка адаптивности, дальнейшая настройка прав доступа и политик БД.

Спасибо за внимание

Ссылки на проект

Сайт	design-folio-practise7.vercel.app
GitHub	github.com/jelr1k/DesignFolio
Макет Figma	figma.com/design/HohoPtvMrGudVojDiC7ROl
Документация API	designfolio-api-bh2q.onrender.com/docs

Открыть сайт на телефоне